

Experiment 2: Deliberative (Goal-Based) Agent — Planning with Search

Aim

To implement a deliberative agent that holds an internal model of a grid world, plans a complete path to its goal with breadth-first search, and then executes the plan.

Theory

A deliberative agent decides by reasoning over an internal world model rather than reacting to the current percept: it searches the space of action sequences for one that reaches the goal, commits to that plan, and executes it. This sense–plan–act architecture contrasts with the sense–react loop of Experiment 1; hybrid architectures layer both, using reactive rules for immediate hazards and deliberation for long-horizon goals.

Breadth-first search over the grid guarantees a shortest path in number of steps. The parent map built during search doubles as the plan: walking it backwards from the goal yields the action sequence. Separating the planning phase (over the model) from the execution phase (in the world) exposes deliberation's central assumption — that the model is correct and the world does not change mid-plan — which the exercises deliberately break.

Algorithm

1. Encode the world as a character grid with start S, goal G and obstacles #; this grid is the agent's internal model.
2. Run BFS from S, recording for each visited cell its parent and the move that reached it.
3. Reconstruct the action sequence by following parents back from G and reversing. 4.

Execute the plan step by step and verify the goal is reached.

```
Planned actions: ['R', 'R', 'D', 'D', 'R', 'R', 'D', 'R', 'R']  
Number of steps: 9
```

Execution:

```
Action R -> (0, 1)  
Action R -> (0, 2)  
Action D -> (1, 2)  
Action D -> (2, 2)  
Action R -> (2, 3)  
Action R -> (2, 4)  
Action D -> (3, 4)  
Action R -> (3, 5)  
Action R -> (3, 6)
```

Goal reached successfully!

Exercises

1. Replace BFS with uniform-cost search where each cell has a terrain cost read from a second grid.

```
[Running] python -u "d:\Agentic AI lab\lab2\uniform_cost_search.py"  
Optimal plan: ['D', 'D', 'R', 'R', 'D', 'R', 'R']  
Minimum total cost: 7
```

2. After planning, block one cell of the path and make the agent detect the failure during execution and re-plan from its current position.

```
[Running] python -u "d:\Agentic AI lab\lab2\tempCodeRunnerFile.py"
Initial plan: ['D', 'D', 'R', 'R', 'D', 'R', 'R']

Cell (3, 2) has been blocked.
Executed D -> (1, 0)
Executed D -> (2, 0)
Executed R -> (2, 1)
Executed R -> (2, 2)

Execution failure at (3, 2)
Re-planning...
New plan: ['L', 'L', 'U', 'U', 'R', 'R', 'R', 'D', 'R', 'D', 'D']
Executed L -> (2, 1)
Executed L -> (2, 0)
Executed U -> (1, 0)
Executed U -> (0, 0)
Executed R -> (0, 1)
Executed R -> (0, 2)
Executed R -> (0, 3)
Executed D -> (1, 3)
Executed R -> (1, 4)
Executed D -> (2, 4)
Executed D -> (3, 4)

Goal reached after re-planning!
```

3. Implement a hybrid agent: reactive rule 'never step into a cell adjacent to a pit' filtering the deliberative plan.

```
[Running] python -u "d:\Agentic AI lab\lab2\hybrid.py"
```

```
Initial plan: ['D', 'D', 'R', 'R', 'D', 'R', 'R']
```

```
Execution:
```

```
Executed D -> (1, 0)
```

```
Executed D -> (2, 0)
```

```
Executed R -> (2, 1)
```

```
Reactive rule BLOCKED action R -> (2, 2)
```

```
Reason: next cell is adjacent to a pit.
```

```
Cell (2, 2) marked as BLOCKED (#)
```

```
New plan from current position: ['L', 'U', 'U', 'R', 'R', 'R', 'D', 'D', 'D', 'R']
```

```
Executed L -> (2, 0)
```

```
Executed U -> (1, 0)
```

```
Executed U -> (0, 0)
```

```
Executed R -> (0, 1)
```

```
Reactive rule BLOCKED action R -> (0, 2)
```

```
Reason: next cell is adjacent to a pit.
```

```
Cell (0, 2) marked as BLOCKED (#)
```

```
New plan from current position: []
```

```
No valid plan exists.
```

```
Goal could not be reached.
```